Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	**Semester:**	1st sem
Name of the Student:	N. Ashok	**Roll Number:**	322126552162
Name of the Lab:	Full Stack	**Timings:**	9:50 to 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	4	3	5	2	14	10	
8/7	2	4	4	5	1	14	10	
15/7	3	4	4	5	1	14	10	
22/7	4	4	4	5	1	14	10	
29/7	5	3	3	5	2	13	10	
5/8	6	4	3	5	1	13	10	
8/8	7	4	3	5	1	13	10	
16/9	8	4	3	5	1	13	10	
23/9	9	4	3	5	1	13	10	
30/9	10	4	3	5	1	13	10	

Name of the Faculty:

Faculty Signature:

Introduction to HTML and CSS.

→ HTML (Hyper text Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading style sheets) is a stylesheet language used to control the Presentation and visual appearance of HTML documents. It is used to define Properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adopt to different screen sizes and devices. CSS can be applied using inline, internal or external style sheets.

→ HTML and CSS work together to develop webpages. Where HTML provides the structure and content, while CSS provides the style, layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive, and user-friendly web interfaces.

Difference between MERN Stack and Full Stack

→ MERN Stack is a javascript - based technology Stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js, MongoDB is used for database management, Express.js handles the backend framework and APIs, React.js is used to develop frontend user interface, and Node.js Provides the server side runtime environment.

→ Full-Stack Development refers the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

1. HTML com
2. HTML tags
3. Headings -
4. Paragraphs
5. Links -
6. Images -
7. Lists - u
8. Tables -
9. Forms -
10. Div and
11. Semanti

→ HTML and CSS work
webpages. where HTML provides the struct
and content. while CSS provides the
layout, visual Presentation. The combi
of both forms the foundation of modern
development and enables the creation of
organized. attractive, and user-frien
web interfaces.

Difference between MERN Stack and Full
→ MERN Stack is a javascript - based
technology Stack used for developing moder
applications. MERN stands for Mongo DB,
Express.js, React.js, Node.js. MongoDB is
for database management, Express.js
handles the backend framework and API
React.js is used to develop frontend use
interface, and Node.js Provides the ser
side runtime environment.

→ Full-Stack Development refers the develo
-ment of both the frontend and backend
web application along with databa
integration. A full Stack developer work
with technologies used for designing the use
interface, implementing server-side logic

HTML components:

1. HTML tags: Define the structure and elements of a webpage.
2. Headings - used to display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag.
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organize data in rows and columns.
8. Forms - used to collect user input using elements such as text boxes, buttons and checkboxes
9. Div and span - used to group and organize webpage content
10. Semantic elements - Elements such as <header>, <nav>, <selection>, <article> <footer>. Provide meaningful webpage structure.

CSS components:

1. Selectors: select HTML elements to apply styles
2. Properties - Define the visual characteristic of elements
3. values - specify the setting of a css Property
4. colors - Define text, background, border colors
5. Fonts - control font family, size, style and weight
6. Backgrounds - set background colors and images
7. Borders - Define the border style, width and radius
8. Margin - controls the space outside an element
9. Padding - controls the space inside element
10. layout - controls the arrangement of elements using Flexbox, Grid and Positioning.

HTML

older
limited
support
use co
for mu
Has few
tags
No <co
Limited
complex
- ratio

CSS VS

Earlier
Basic S
Limited
No flex
Limited
No medi
Uses a si

CSS components:

1. Selectors: Select HTML elements to apply sty
2. Properties - Define the visual characteristic of elements
3. values - specify the setting of a CSS Proper
4. colors - Define text, background, border co
5. Fonts - control font family, size, style an weight
6. Backgrounds - set background colors and i
7. Borders - Define the border style, width and radius
8. Margin - controls the space outside an element
9. Padding - controls the space inside elem
10. layout - controls the arrangement of elements using Flexbox, Grid and Positioning.

HTML VS HTML5

HTML	HTML5
older version of HTML	Latest major version of HTML
limited multimedia support	supports audio and vedio
use external Plugins for multimedia	No external plugs needed
Has fewer semantic tags	Provides semantic tags
No <canvas> element	supports <canvas>
Limited web storage	supports local storage
complex DOCTYPE declaration.	simple <!DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features	Provides advanced styling features.
Limited animations	supports animations and transitions.
No flexible layout systems	supports flexbox and grid
Limited visual effects	supports shadows, gradients, effects
No media queries	supports media queries
uses a single specification	Divided into different modules.

HTML VS HTML5

HTML	HTML5
older version of HTML	Latest major version of HTML supports audio and vedio
limited multimedia support	
uses external Plugins for multimedia	No external plugs needed
Has fewer semantic tags	Provides semantic tags
No <canvas> element	supports <canvas>
Limited web storage	supports local storage
complex DOCTYPE declaration.	simple <!DOCTYPE html>

CSS VS CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features	Provides advanced styling features.
Limited animations	supports animations and transitions.
No flexible layout systems	Supports flexbox and grid
Limited visual effects	supports shadows, gradients, effects
No media queries	supports media queries
Uses a single specification	Divided into different

Design a static web Page using HTML and CSS
Pomodoro Timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body{
            font-family: Arial;
            background: #f5f5f5;
            text-align : center;
            Padding: 50 Px;
        }
        .timer{
            width: 300Px;
            margin : auto;
            Padding: 30 Px;
            background : white;
            border-radius : 15 Px;
            box-shadow: 0 0 10Px grey;
        }
        h1{
            color : #e74c3c;
        }
        .time{
            font-size: 50 Px;
            margin: 20Px;
        }
        button{
            Padding: 10Px 20Px;
            margin : 5Px;
            border : none;
```

```
    </style>
</head>
<body>
</body>
</html>
```

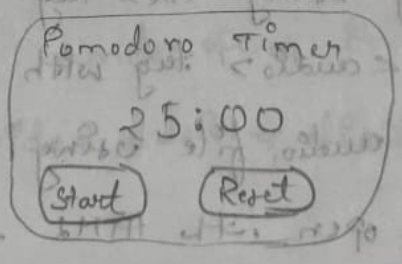
O/P:

HTML components:

1. HTML tags: Define the structure and elements of a webpage.
2. Headings - used to display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag.
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organize data in rows and columns.
8. Forms - used to collect user input using elements such as text boxes, buttons and checkboxes
9. Div and span - used to group and organize webpage content
10. Semantic elements - Elements such as <header>, <nav>, <selection>, <article> <footer>. Provide meaningful webpage structure.

```
    border-radius: 5px;
    background: #e74c3c;
    color: white;
  }
</style>
</head>
<body>
  <div class="timer">
    <h1>Pomodoro Timer</h1>
    <div class="time">25:00</div>
    <button>Start</button>
    <button>Reset</button>
  </div>
</body>
</html>
```

o/p:



Design a static web page using HTML and CSS
Pomodoro Timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body{
            font-family: Arial;
            background: #f5f5f5;
            text-align: center;
            Padding: 50 Px;
        }
        .timer{
            width: 300Px;
            margin: auto;
            Padding: 30 Px;
            background: white;
            bőder-radius: 15 Px;
            box-shadow: 0 0 10Px grey;
        }
        h1{
            colő: #e74c3c;
        }
        .time{
            font-size: 50 Px;
            margin: 20Px;
        }
        button{
            Padding: 10Px 20Px;
            margin: 5Px;
```

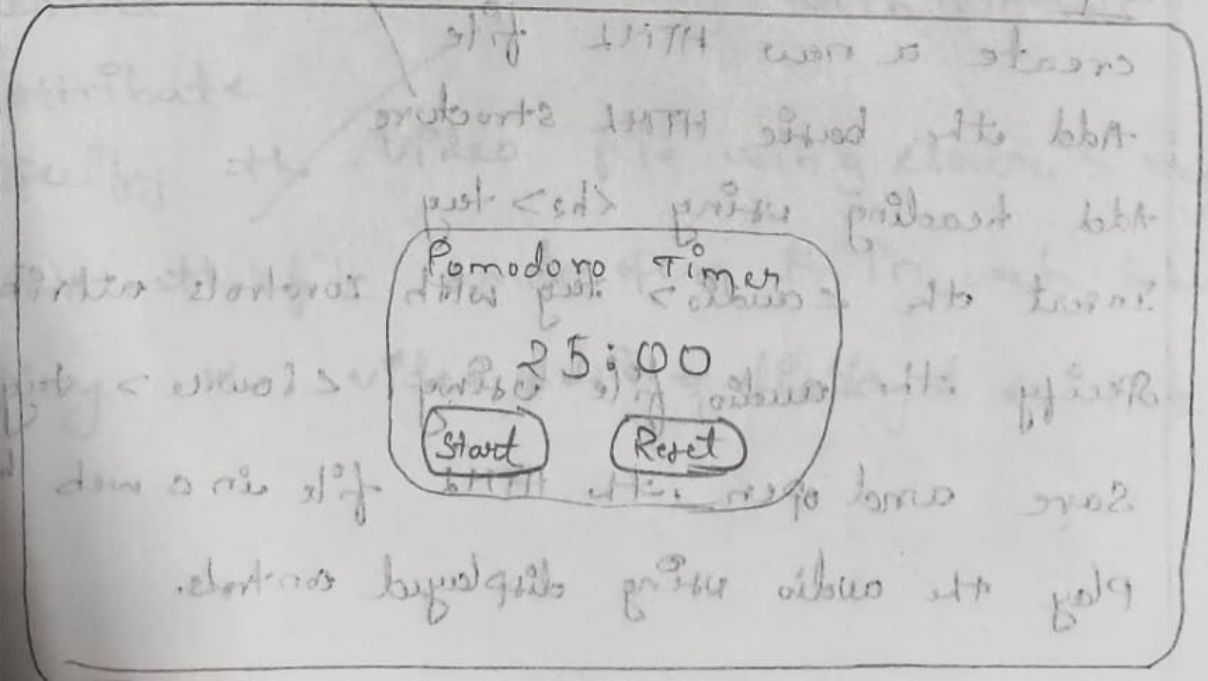
Introduction to HTML and CSS.

→ HTML (Hyper text Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading style sheets) is a stylesheet language used to control the Presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adopt to different screen sizes and devices. CSS can be applied using inline, internal or external stylesheets.

```
        border-radius: 5px;
        background: #e74c3c;
        color: white;
    }
</style>
</head>
<body>
    <div class="timer">
        <h1>Pomodoro Timer</h1>
        <div class="time">25:00</div>
        <button>Start</button>
        <button>Reset</button>
    </div>
</body>
</html>
```

O/P:



1) Design an HTML code using audio tags.

code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Audio Example</title>
</head>
<body>
    <h2> my favourite song </h2>
    <audio controls>
        <source src="song.mp3"
        type="audio/mpeg">
        Your browser does not support
        the audio tag.
    </audio>
</body>
</html>
```

O/P: Displays an audio player with play, Pause and volume controls.

Steps:

create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls attribute
Specify the audio file using <source> tag
Save and open the HTML file in a web browser
Play the audio using displayed controls.

ii) Design a HTML

code:

```
<!DOCTYPE html
<html>
<head>
    <title>
</head>
<body>  <h2>
    <vedio
    <s
</vide
</body>
</html>
```

O/P: Displays a
volume and fu

Steps:

create a new
Add a basic
Add a headi
Insert the
attribute
specify the
Save the file
play the v

1) Design an HTML code using audio tags.

code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Audio Example</title>
</head>
<body>
    <h2> my favourite song </h2>
    <audio controls>
        <source src="song.mp3"
        type="audio/mpeg">
        Your browser does not support
        the audio tag.
    </audio>
</body>
</html>
```

O/P: Displays an audio player with play, Pause and volume controls.

Steps:

create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls attri
Specify the audio file using <source> ta
Save and open the HTML file in a web
Play the audio using displayed controls.

ii) Design a HTML code using vedio tags:-

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body> <h2> my vedio </h2>
    <vedio width = "400" controls >
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

Steps:

create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with .controls attribute
specify the video file using <source> tag
save the file and open it in web browser
play the video using displayed controls.

ii) Design a HTML code using video tags:

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> my vedio </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

Steps:

create a new HTML file

Add a basic HTML structure

Add a heading using the <h2> tag

Insert the <video> tag with controls attribute

specify the video file using <source> tag

Save the file and open it in web browser

play the video using displayed controls.

ii) Design a HTML code using Vedio tags:

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> my vedio </h2>
    <vedio width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

Steps:

create a new HTML file

Add a basic HTML structure

Add a heading using the <h2> tag

Insert the <video> tag with controls attribute

specify the video file using <source> tag

save the file and open it in web browser

play the video using displayed controls.

ii) Design a HTML code using, vedio tags:

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> my vedio </h2>
    <vedio width = "400" controls >
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

Steps:

create a new HTML file

Add a basic HTML structure

Add a heading using the <h2> tag

Insert the <video> tag with controls attribute

specify the video file using <source> tag

Save the file and open it in web browser

Play the video using displayed controls.

write a Program using Javascript (arrays, functions)

code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

Function displayarray(arr){
    console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findsum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++)
        sum+=arr[i];
    }
    return sum;
}
```

2. Develop dyna

Program:

```
<!DOCTYPE
<html>
<head>
<ti
<sty
```

write a Program using Javascript (arrays, functions)

code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

Function displayarray(arr){
    console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findsum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++)
    sum+=arr[i];
}
    return sum;
```

EXPERIMENT-04

1. Develop dynamic webpage using java script

Program:

```
<!DOCTYPE html>
<html>
<head>
  <title> Dynamic webpage </title>
  <style>
    body{
      text-align: centre;
      font-family: Arial;
      margin-top: 100px;
    }
    button{
      Padding: 10px 20px;
      fontsize: 16px;
      cursor: pointer;
    }
  </style>
</head>
<body>
<h1 id="heading"> welcome to my
    webpage </h1>

<p id="message"> click the button to change
    this webpag </p>
```

EXPERIMENT-04

Q. Develop dynamic webpage using java script.

Program:

```
<!DOCTYPE html>
<html>
<head>
<title> Dynamic webpage </title>
<style>
    body{
        text-align : centre ;
        font-family : Arial;
        margin-top : 100 Px ;
    }
    button {
        Padding : 10 Px 20 Px ;
        font-size : 16 Px ;
        cursor : Pointer ;
    }
</style>
</head>
<body>
<h1 id = "heading"> welcome to my
    webpage </h1>
<p id = "message"> click the button to change
    this webpage </p>
```

DA INSTITUTE OF
GY & SCIENCES
meil
AUTONOMOUS)
by Andhra Universit
CSE, IT, Mech, Civil
Bheemunipatnam M
: 226395, Website:

```
<button onclick = "changePage()" > click me
                                  </button>

<script >
    function changePage(){
    document.getElement by Id ("heading")
    inner HTML = "welcome to javascript!";
    document. getelement by Id ("heading")
        inner HTML = "welcome to javascript!"
    document.getElement BY Id ("message")
        innerHTML = "the webpage has been
    changed dynamically" ;
    document. body style. background color =
                                "lightblue";
    }
</script>
</body>
</html>
```

output :-

• when th
function
heading
also cha

Th

```
<button onclick = "changePage()" > click me
</button>

<script>
    function changePage(){
    document.getElementById("heading")
    .innerHTML = "welcome to javascript
    document.getElementById("heading")
    .innerHTML = "welcome to javascript!
    document.getElementById("message")
    .innerHTML = "the webpage has been
    changed dynamically";
    document.body.style.backgroundColor =
                "lightblue";
    }
</script>
</body>
</html>
```

output :-

Welcome to my webpage

click the button to change the webpage

[clickme]

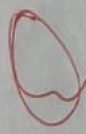
• When the user clicks "clickme", the javascript function changepage() is called, then the heading dynamically changes and background also changes.

After clicking the button

welcome to javascript !

The webpage has been changed dynamically

[click ok]



Welcome to my webpage

click the button to change the webpag

[clickme]

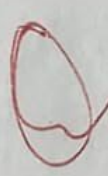
• When the user clicks "clickme", the javascrip function changePage () is called, then the heading dynamically changes and background also changes.

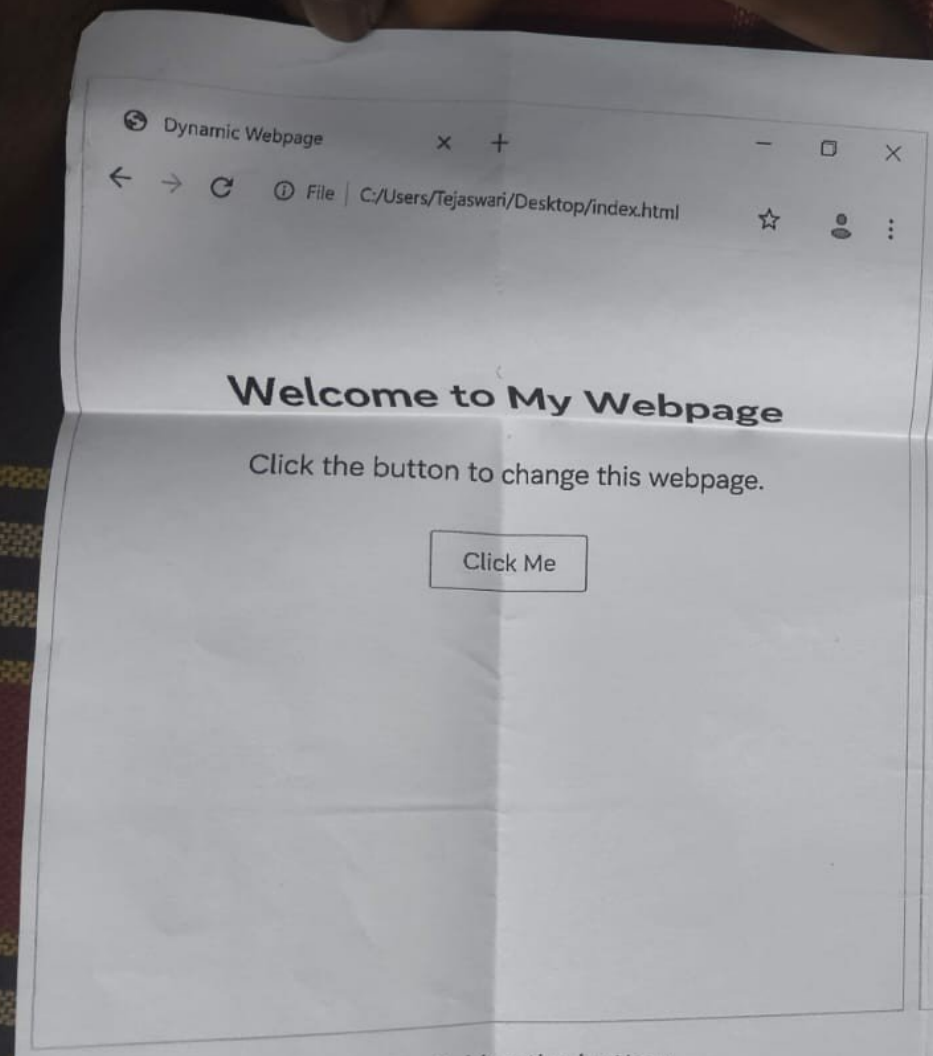
After clicking the button

welcome to javascript !

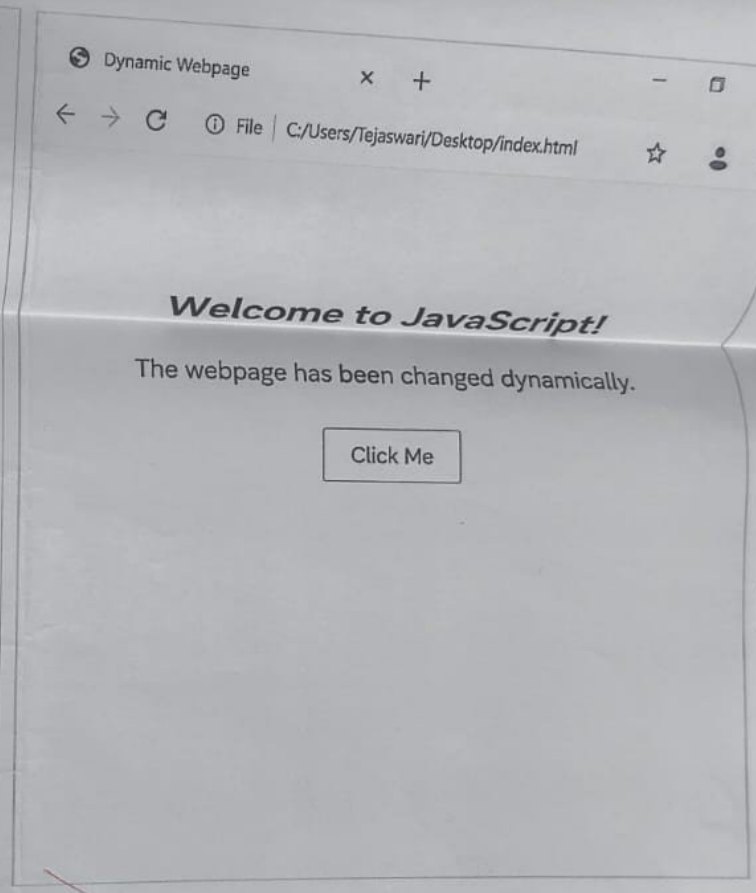
The webpage has been changed dynamically

[click me]

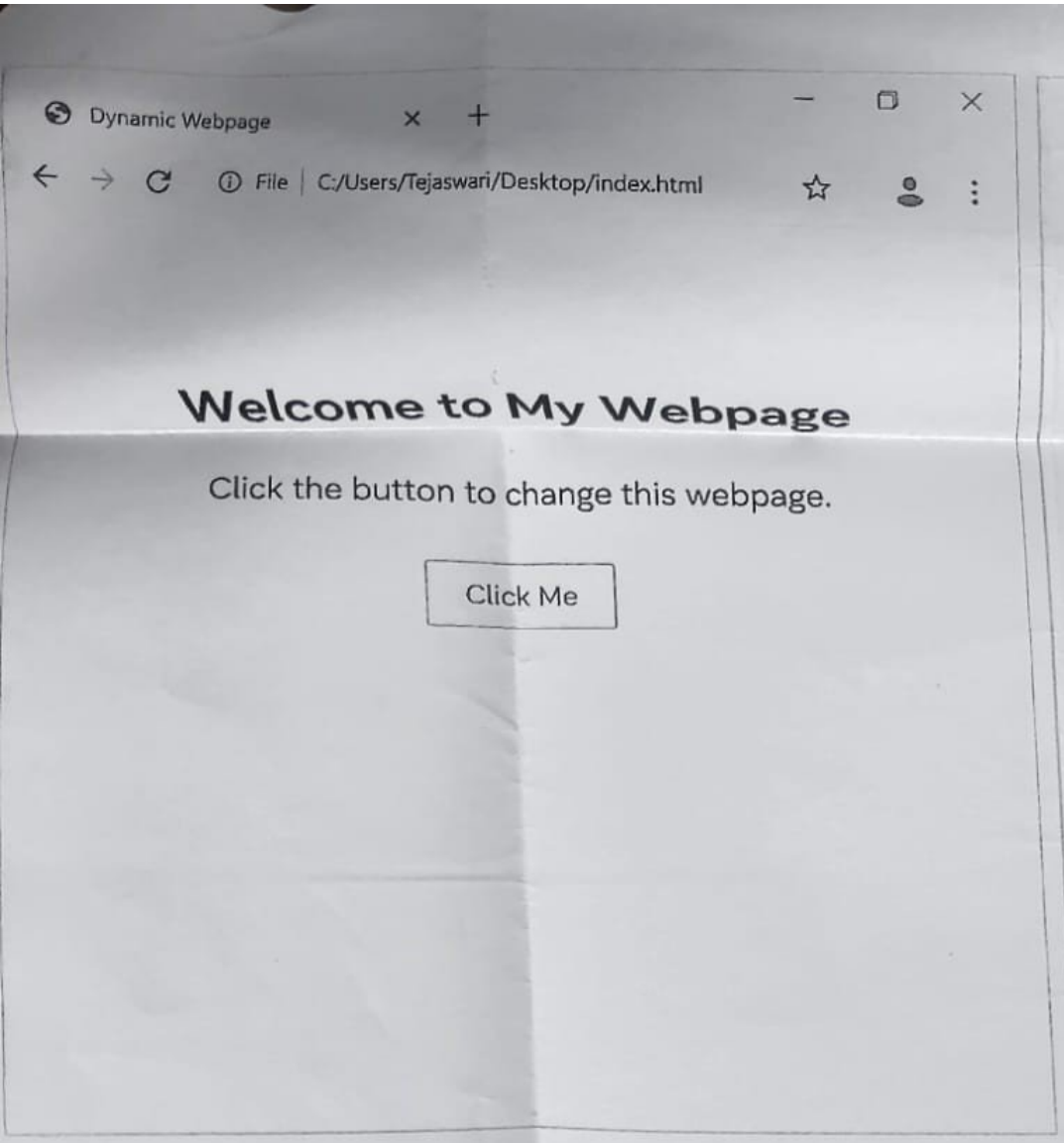




Before clicking the button



After clicking the button



Before clicking the button



After clicking the button

EXPERIMENT - 05

5 Implement modules in Node.js

In Node.js you can implement modules using two distinct systems. common JS (JS) which is traditional default, and ES modules (ESM) the modern javascript standard.

1. the common JS method (default):

common JS uses module.exports to share code and require() to import it. this system loads modules synchronously.

create the module (math.js):

```
const add = (a, b) => a+b
const subtract = (a, b) => a-b
module.exports = {
    add,
    subtract
};
```

b use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

output: 15
5

2. The Es modules method (modern standard):

Es modules use export to share code and import to use it. To make node.js recognize Es modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercalc";
export default calculatorName;
```

b. use the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

output: supercalc

3. using built-in cor

Node.js also comes
of built-in modules (li
& path for file routin
installation.

we will load ther

```
// comman
const fs =
// Es modu
import fs
```

Here are some of

ones:

1. fs = file
2. http = HTTP
3. path - file
4. os - oper
5. events - E
6. util - ut
7. stream -
8. crypto -
9. url - U
10. query

3. Using Built-in core modules:

Node.js also comes equipped with dozens of built-in modules (like fs for file systems or path for file routing) that don't need installation.

We will load them by module names:

```
// common js style
const fs = require('fs');
// ES modules style
import fs from 'fs';
```

Here are some of the most commonly used ones:

1. fs = file system operations
2. http = HTTP server and client
3. path - file path utilities
4. os - operating system utilities
5. events - event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL query string handling

Example: Simple HTTP server

```
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end("Hello World!");
}).listen(8080);
```

output: Hello world!

6) Develop mini

Node.js

- Simple mini
- Student Mana
  1. Add
  2. Vie
  3. D
  4. Or
  5. E
- Project str

server.js

```
const exp
const app
const POR
let stu
app.u
app.use
app g
```

Example: Simple HTTP server

```
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end("Hello World!");
}).listen(8080);
```

output: Hello World!

EXPERIMENT - 6

6) Develop mini application using express and Node.js.

• Simple mini application using node.js + Express.js

Student Management System:

1. Add a student
2. View all students
3. delete a student
4. Use Express.js router
5. Simple HTML interface

Project structure :-

```
student-app/
├── server.js
└── Public
    └── index.html
```

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("Public"));
app.get("/students", (req, res) => {
    res.json(students);
});
```

6) Develop mini application using express and Node.js.

- simple mini application using node.js + Express.js

Student Management system:

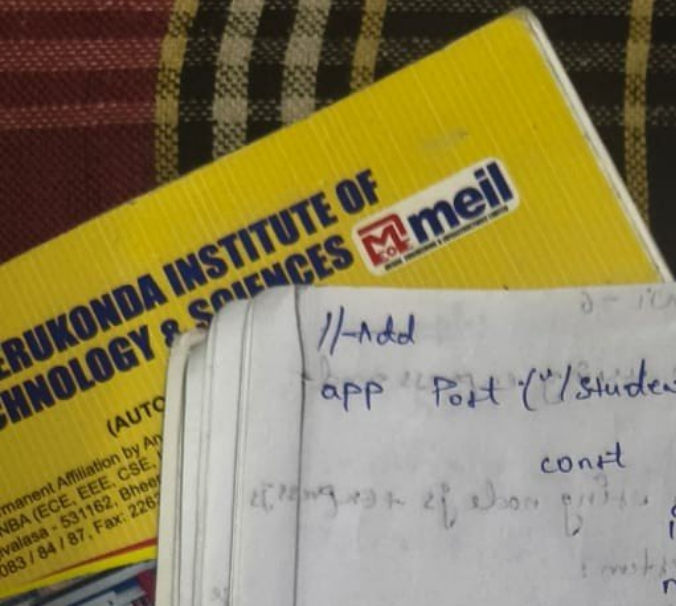
1. Add a student
2. View all students
3. delete a student
4. Use Express.js router
5. Simple HTML interface

Project structure :-

```
student-app/
├── server.js
└── Public
    └── index.html
```

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("Public"));
app.get("/students", (req, res) => {
    res.json(students);
});
```




```
//-Add
app Post ("/students", (req, res) => {
    const student = {
        id: students.length+1,
        name: req.body.name,
        course: req.body.course
    };
    students.Push (student);
    res.json ({ message: "student added
                successfully",
                student: student });
});

// Delete
app.delete ("/students/:id", (req, res) =>{
    const id = Parse Int (req.Params.id);
    students = students.filter (student =>
        student.id !== id);
    res.json ({
        message: "Student deleted Successfully"});

// Start server
app.listen (PORT, () => {
    console.log (`Server running at http://
        localhost: ${PORT}`);
```

```
});
Public /index.html
<!DOCTYPE html>
<html>
<head>
    <title>
</head>
<body>
    <h1> Stu
    <input ty
    Placehold
    <input ty
    Placehold
    <button
        Add
    <h2> St
    <ul id=
<script>
    funct
    let
    let co
```

});

Public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
</head>
<body>
    <h1> Student management system </h1>
    <input type="text" id="name"
    Placeholder=" Enter student name">
    <input type="text" id="course"
    Placeholder=" Enter course ">
    <button onclick="addStudent()">
        Add student </button>
    <h2> Student list </h2>
    <ul id="studentlist"> </ul>
<script>
    function addStudent(){
    let name= document.getElementBy
        Id("name").value;
    let course =document.getElement
        ById("course").value;
```

```
fetch("/students", {
    method: "POST",
    headers: {
        "content-Type": "applic
                        (j.son"},
    body: JSON.stringify({
        name: name,
        course: course
    })
})
.then(response => response.json
.then(data => {
    alert(data.message
document.getElementById("name").value
document.getElementById("course").value
    displayStudents();
});
}
function displayStudents() {
    fetch("/students").then(respo
        response.json()).th
        (students => {
    let list = document.getElementBy
```

```
list.innerHTML = "";
students.forEach(student => {
const li =
document.createElement("li");
li.innerHTML = student.id + " - " +
                student.name + " - " +
                student.course + " <button
onclick='deleteStudent(" + student.id + ")'>
                Delete</button>";
list.appendChild(li);
        });
    });
}
function deleteStudent(id) {
    fetch("/students/" + id,
        {method: "DELETE"})
.then(response => response.json())
.then(data => {
        alert(data.message);
        displayStudents();
    });
}
displayStudents();
</script>
</body>
</html>
```

output :

Student management system

Enter student name

Enter course

Add Student

Student List :-

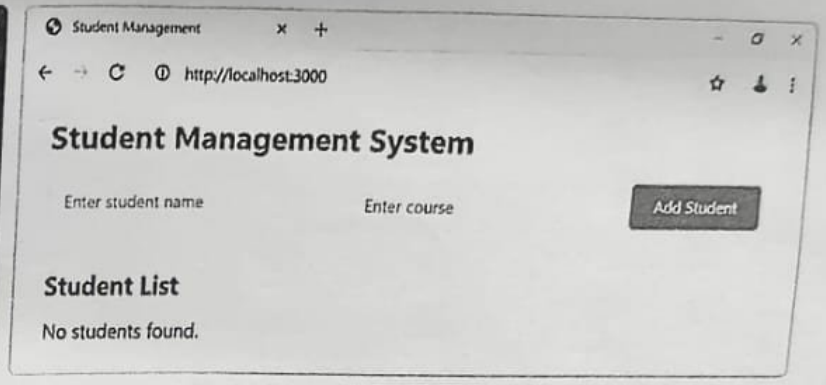
1. Manohar – CSE delete
2. Manoj – CSE delete

the Express.js server is successfully started at http://localhost:3000

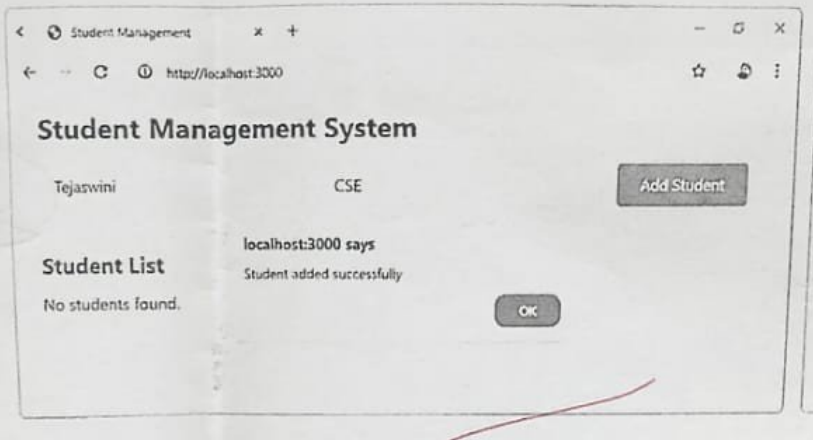
the browser displays the student management system with fields.



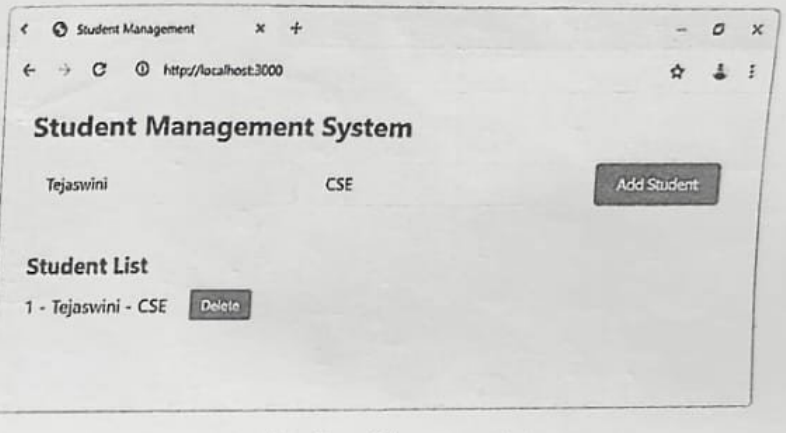
1. Terminal Output (after running `node server.js`)



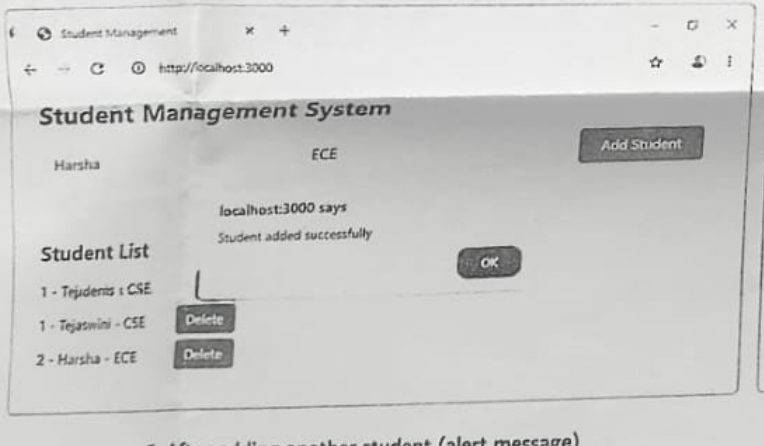
2. Initial Output in Browser (no students)



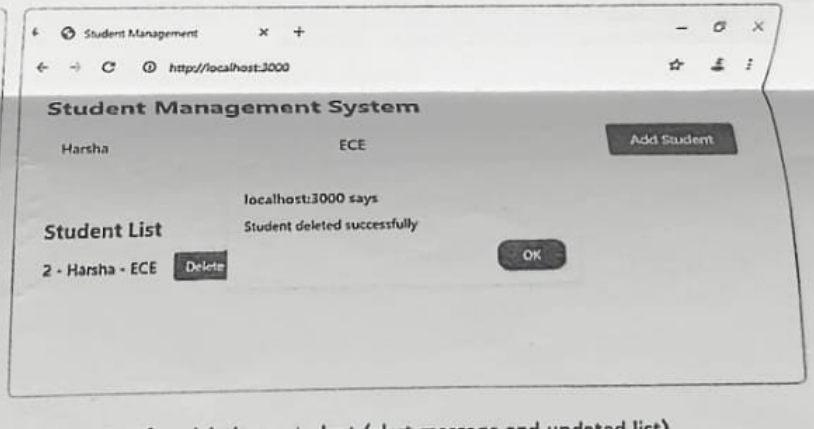
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

EXPERIMENT -7

7. Create a Functional component and class component using react.

Aim: To develop a react application using Functional components and class components to display Student details, manage state, handle button clicks and dynamically update data on a web

- Functional component:

A Functional component is a javascript function that returns JSX to display the UI. Interface. It uses react Hooks such as useState to manage state and update the webpage dynamically.

Program:

```
import react, {useState} from
'react':
function StudentFunctional(){
    const [name, setName] = useState("Ashok");
    const [course, setCourse] = useState("CSE");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Ashokk");
    };
```

```
const changeCourse = () => {
    setCourse("CSD");
};
const incrementCount = () => {
    setCount(count++);
};
return (
  <div style={{
        border: "8px solid blue",
        Padding: "20px";
        margin: "20px,
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1>Student Details</h1>
    <h2>Functional Component</h2>
    <p><b>Name:</b>{name}</p>
    <p><b>Button clicks:</b>{count}</p>
    <button onClick={changeName}>
        changeName</button>
    <button onClick={changeCourse}>
        change course</button>
    <button onClick={incrementCount}>
        Increment</button>
  </div>
```

UKONDA INSTITUTE OF
NOLOGY & SCIENCES
meil
(AUTONOMOUS)
& Approved by AICTE,
hemical) & NAAC with 'A+' Grade)
Bhapatnam District.
mail: principal@anits.edu.in

output: When the user it the buttons, the name course and count change dynamically without reloading the webpage.

· class component:

A class compo
class that exten
the render() m
managers state
setState().

Program:

import React
'react':
class Studentde
Constructo
sup
this

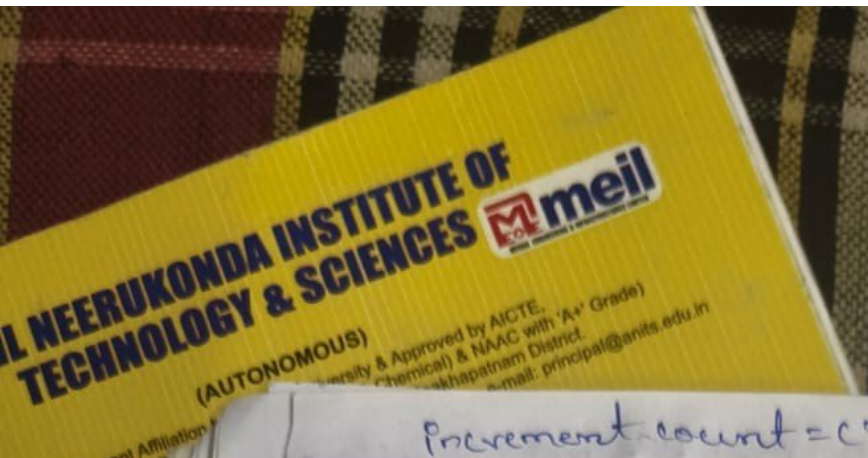
ch

. class component:

A class component is an ES6 Javascript class that extends React.component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React {component} from
'react';
class studentclass extends component {
    constructor (Props){
        super (Props);
        this.state = {
            name = "Ashok",
            course = "computer science"
            count = 0
        };
    }
    changeName = () => {
        this.setState ({
            name: "Ashokk"
        });
    };
    changeCourse = () => {
        this.setState ({
            course: "CSM"
        });
};
```



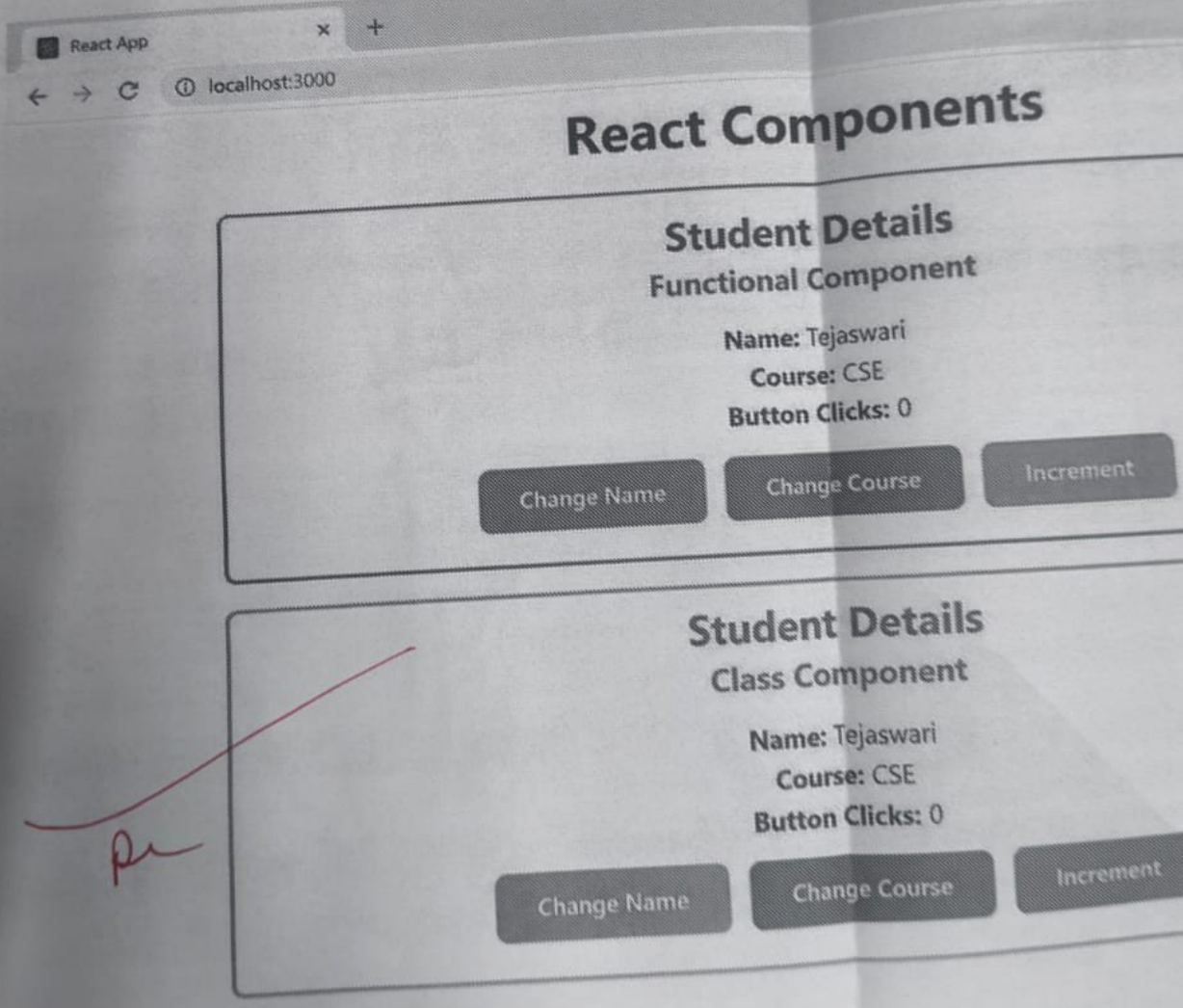

```
increment count = () => {
    this.setState((PrevState) => {
        count: PrevState.count + 1
    });
};
render() {
    return (
        <div style={{
            border: "2Px solid green",
            Padding: "20Px",
            margin: "20Px",
            borderRadius: "10Px",
            textalign: "center"
        }}>
        <h1> Student details </h1>
        <h2> Class component </h2>
        <P> <b> Name: </b> {this.state.name} </P>
        <P> <b> Course: {this.state.course} </P>
        <P> <b> Button clicks: </b> {this.state.count}
        <button onClick={this.changeName}>
            changeName </button>
        <button onClick={this.changeCourse}>
            change course </button>
```

<button on

output:

<button onclick={this.incrementcounts}>
Increment </button>

output:



ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

(AUTONOMOUS)

ANITS

EXPERIMENT - 08

8. Implement HTTP methods using JSON server

Aim: To Implement HTTP methods such as (POST, PUT, PATCH and DELETE using JSON server to Perform CRUD (Create, Read, Update, Delete) operations on student records.

Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to Perform HTTP operations without developing a backend server from scratch. HTTP methods are used to communicate between a client and a server

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record the server

a. Create a JSON server

Step1: Initialize the Project

open the VS code terminal and execute the following commands.

EXPERIMENT - 08

8. Implement HTTP methods using JSON server

Aim: To implement HTTP methods such as (POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update, Delete) operations on student records.

Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch. HTTP methods are used to communicate between a client and a server

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record the server

a. Create a JSON server

Step1: Initialize the Project

open the VS code terminal and execute the following commands.

IL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES meil
(Permanent Affiliation)
ccredited by NBA (ECE, EE
Sangivalasa - 5311
s: 08933-225083 / 84 / 87,
NITE, 'A+' Grade)
e.edu.in

Step 3: Start the JSON server

For the current JSON server version, enter

npx json-server db.json

The server start at:

http://localhost:3000

The student records can be accessed at

http://localhost:3000/students

2. Implementation of HTTP methods

a. GET method: The GET method is used to retrieve student records from JSON server

Code:

```
fetch ('http://localhost:3000/students')
.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log(error);
});
```

output:

b. POST method: The

a new student

Code: fetch ('ht

method

header

'conten

},

body: J

no

co

m

})

})

.then(

.then

})

output:



B. POST method: The POST method is used to insert a new student record into the server.

code:

```
fetch ('http://localhost:3000/students',{
    method: 'POST',
    headers {
    'Content-Type':'application/json'
    },
    body.JSON.stringify({
        name: 'Rahul';
        course: 'CSE'
        marks : 92
    })
})
.then(response => response.json())
.then(data =>{
        console.log(data);
})
```

output:

B. POST method: The POST method is used to insert a new student record into the server.

code:

```
fetch ('http://localhost:3000/students',{
    method:'POST',
    headers {
    'content-Type':'application/json'
    },
    body.JSON.stringify({
        name:'Rahul';
        course:'CSE'
        marks:92
    })
})
.then(response => response.json())
.then(data =>{
        console.log(data);
})
```

```
.catch (error => {
        console.log(error);
});
```

output:

c. PUT Method: the PUT method is used to repla
on existing student record with update d

code:
```
fetch ('http://localhost:3000/students/
method : 'PUT',
headers : {
    'content-type' : 'application/json,
},
body : JSON.stringify({
    name : 'Ashok',
    course : 'AIML',
    marks : 95
})
})
.then (response => response.json())
```

```
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:

D. PATCH Method: the PATCH method is used to update only selected fields of an existing student record.

code:

```
fetch('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'content-Type': 'application/json',
    },
    body: JSON.stringify({
        marks: 96
    })
})
.then(response => response.json())
.then(data => {
```

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

(AUTONOMOUS)

ANITS

output:

1. GET All Students (http://localhost:3000/students)

localhost:3000/students

```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```

2. GET Student by ID

localhost:3000/students/68dca123456789abcdef1234

```
{
  "_id": "68dca123456789abcdef1234",
  "name": "Rahul",
  "course": "CSE",
  "marks": 92,
  "__v": 0
}
```

3. POST (Create Student) – Using Postman

localhost:3000/students

```
{
  "name": "Tejaswari",
  "course": "AIML",
  "marks": 95,
  "_id": "68dcf9b7e4c9e8e1f3a6c5678",
  "__v": 0
}
```

Status: 201 Created Time: 48 ms Size: 160 B

4. PUT (Update Student) – Using Postman

localhost:3000/students/68dca123456789abcdef1234

```
{
  "_id": "68dca123456789abcdef1234",
  "name": "Rahul",
  "course": "AIML",
  "marks": 96,
  "__v": 0
}
```

Status: 200 OK Time: 62 ms Size: 165 B

5. DELETE (Remove Student) – Using Postman

localhost:3000/students/68dca123456789abcdef1234

```
{
  "message": "Student deleted successfully"
}
```

Status: 200 OK Time: 45 ms Size: 46 B

6. GET All Students (After Deletion)

localhost:3000/students

```
[]
```

Status: 200 OK Time: 28 ms Size: 2 B

output:

After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be:

output:

Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server the CRUD operations were performed on student records the data has retrieved, inserted, updated and deleted using HTTP requests.

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE.
Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with 'A+' Grade)
Sangivalasa - 531162, Bheemunipatnam Mandal, Visakhapatnam District.
Phones: 08933-225083 / 84 / 87, Fax: 226395, Website: www.anits.edu.in, e-mail: principal@anits.edu.in
ANITS
meil

EXPERIMENTS-9

9. Develop CRUD operations using MongoDB

Aim: To develop a web application using Node.js Express.js and MongoDB to perform CRUD operations (Create, Read, Update, Delete) on Student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON-like documents. ∴ CRUD operations are used to manage data in a database.

Mongoose in an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collection and perform database operations.

1. Installation and Project setup:

```
Mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm -init -y
npm install express mongoose
```

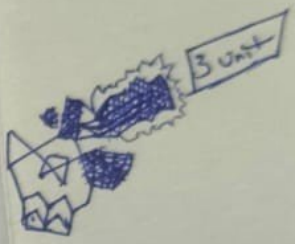
Program: server.js

```
const express = require('
const mongoose = require
const app = express();
const PORT = 3000;
app.use(express.json(
mongoose.connect(
.then(() => console.
.catch(err => console.
// Create Student
const Student = sch
mongoose.schem
name: {
type:
requi
},
course: {
type:
requ
},
marks: {
typ
req
```

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());

  mongoose.connect('mongodb://127.0.0.1:27017/
                                    college')
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.log(err));

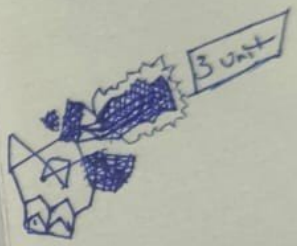
// create student
const Student schema = new
mongoose.Schema({
    name: {
      type: String
      required: true
    },
    course: {
      type: String,
      required: true
    },
    marks: {
      type: Number
      required: true
```



```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());

  mongoose.connect('mongodb://127.0.0.1:27017/college')
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));
//create.Student
const StudentSchema = new
mongoose.Schema({
      name: {
        type: String
        required: true
    },
    course: {
        type: String,
        required: true
    },
    marks: {
        type: Number
        required: true
```

```
Student.find():
res.status(500).json({message: error.message});
});
// READ: Get student by ID
app.get('/students/:id', async (req, res) => {
  try {
    const student = await
    Student.findById(req.params.id);
    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student)
  } catch (error) {
    res.status(500).json({ message:
      error.message });
  }
});
// update
app.put('/student/:id', async (req, res) => {
  try {
```



ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE)
Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with 'A+' Grade)
Sangivalasa - 531162, Bheemunipatnam Mandal, Visakhapatnam District
Phones: 08933-225083 / 84 / 87, Fax: 226395, Website: www.anits.edu.in, e-mail: principal@anits.edu.in
ANITS
meil

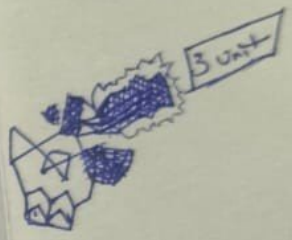
```
const student = await
Student findByAndUpdate(
    req.Params.id,
    req.body
    {new: true, runValidators: true}
);
if (!student){
    return res.Status(404).json({
    message: "Student not found"
    })
}
res.Status(200).json(student);
} catch (error){
    res.status(400).json({message
    error.message});
}
});
//DELETE
app.delete('/students/:id', async(req,res)=>{
    try{
    const student await student
        findBYID Delete(
```

```
req.Params.id
);
if (!student){
    return res.s
    message:"
});
}
res.Status(200
    messag
});
} catch (er
    res.stat
});
```

output: R

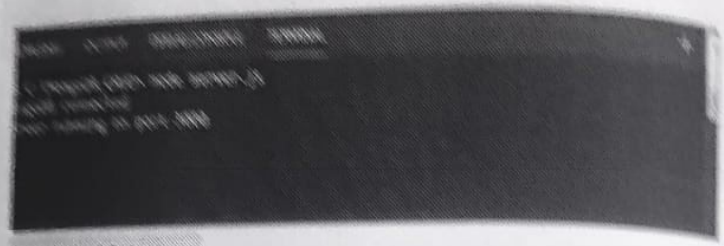
```
req.params.id
);
if (!student) {
    return res.status(404).json({
        message: "student not found"
    });
}
res.status(200).json({
    message: "student deleted
              successfully"
});
} catch (error) {
    res.status(500).json({ message:
        error.message });
}
});
```

output:

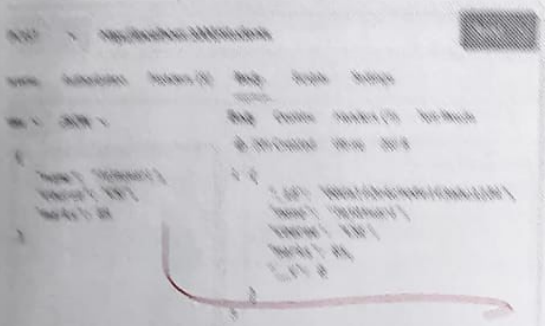


```
    req.Params.id
  );
if (!student) {
    return res.status(404).json({
      message: "student not found"
    });
  }
  res.status(200).json({
      message : "student deleted
                successfully"
    });
} catch (error) {
    res.status(400).json({ message:
                error.message });
}
});
```

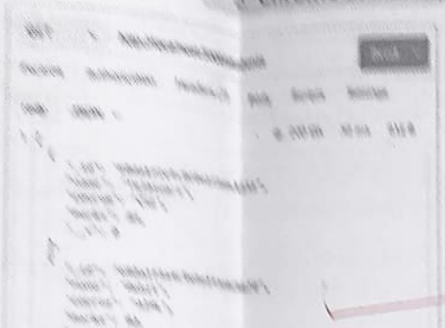
output:



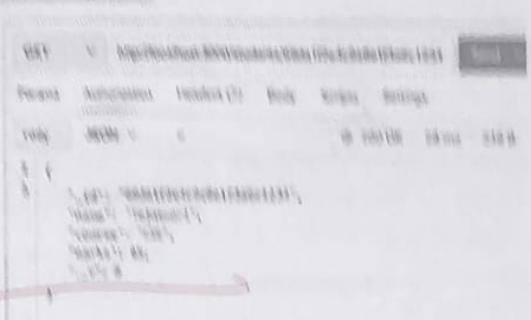
2. Chrome Browser (Home Page)



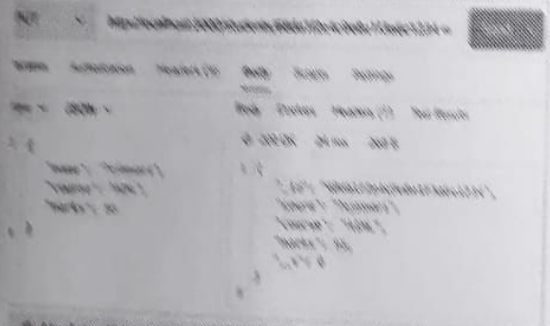
3. Create (POST) – Add Student



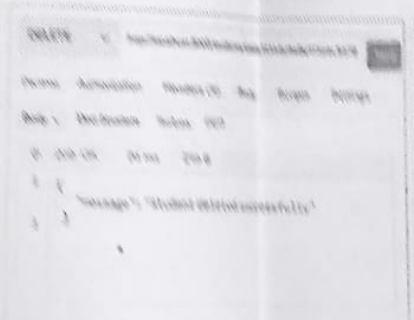
4. Read (GET) – Get All Students



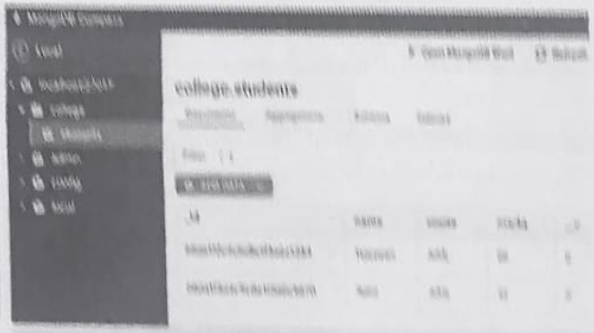
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View

```
const student = await
student findByAndUpdate(
    req.Params.id,
    req.body
    {new: true, runValidators: true}
);
if (!student){
    return res.Status(404).json({
     message: "Student not found"
    })
}
res.status(200).json(student);
} catch (error){
    res.status(400).json({message:
    (error.message})
}
});
//DELETE
app.delete('/students/:id', async(req,res)=>
    try{
    const student await student
        findBYIDDelete(
```

```
Student.find():
res.status(500).json({message: error.message});
}
});
// READ: Get student by ID
app.get('/students/:id', async (req, res) => {
  try {
    const student = await
    Student.findById(req.params.id);
    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student)
  } catch (error) {
    res.status(500).json({ message:
      error.message });
  }
});
// update
app.Put('/student/:id', async (req, res) => {
  try {
```

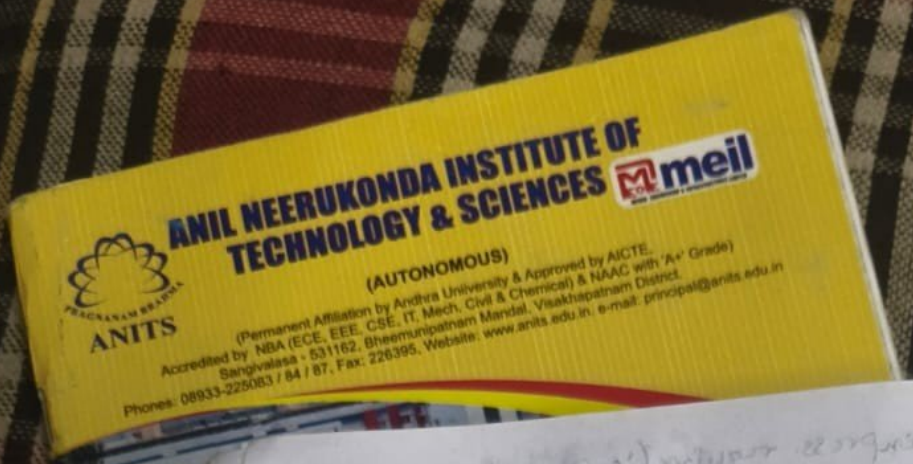
```
});
// Create Student model
const Student =
    mongoose.model('Student', StudentSch

// CREATE: Add
app.post('/Students', async (req, res)=
    try {
        const student = new Student(req.b
        const saved Student = await Student
                                    savel

    res.status(201).json(saveStudent);
    } catch (error) {
        res.status(400).json({ message:
            error.message });
    }
});

// READ: Get all Students
app.get('/Students', async (req, r

    try {
        const Student = await
```




```
    }
  });

// Create Student model
const Student =
  mongoose.model('Student', StudentSchema);

// CREATE: Add
app.post('/Students', async (req, res) => {
  try {
    const student = new Student(req.body);
    const savedStudent = await student
                                .save();
    res.status(201).json(savedStudent);
  } catch (error) {
    res.status(400).json({ message:
        error.message });
  }
});

// READ: Get all Students
app.get('/Students', async (req, res) =>
  try {
    const student = await
```

```
    Student.find();
    res.status(500).json({
  }
});
// READ: Get student by
app.get('/students/:
  try {
    const student
    Student.find
    if (!student)
      return res
      message:
    });
  }
  res.status(20
} catch (error) {
  res.status
  error.mes
  }
});
// update
app.put
  try {
```

1. Develop CRUD operations using Mongo DB

Aim: To develop a web application using Node.js Express.js and Mongo DB to perform CRUD operations (Create, Read, Update, Delete) on Student records.

Theory: Mongo DB is a NoSQL database that stores data in flexible, JSON-like documents. CRUD operations are used to manage data in a database.

Mongoose in an Object Data Modeling (ODM) library for Mongo DB and Node.js. It helps define schemas, interact with collection and perform database operations.

1. Installation and Project setup:

```
Mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm -init -y
npm install express mongoose
```

Program: Server.js

output:

After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be:

output:

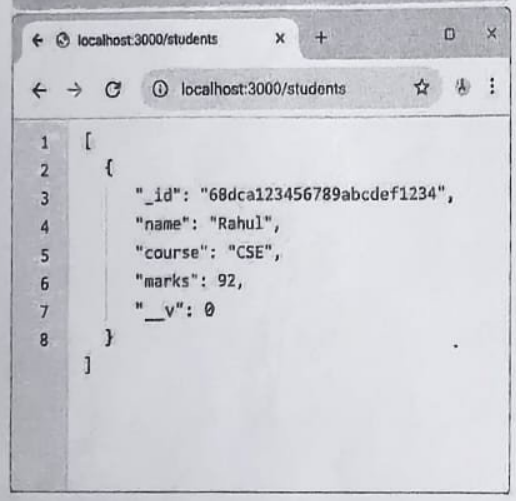
Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server the CRUD operations were performed on student records the data has retrieved, inserted, updated and deleted using HTTP requests.

• console.log(data);

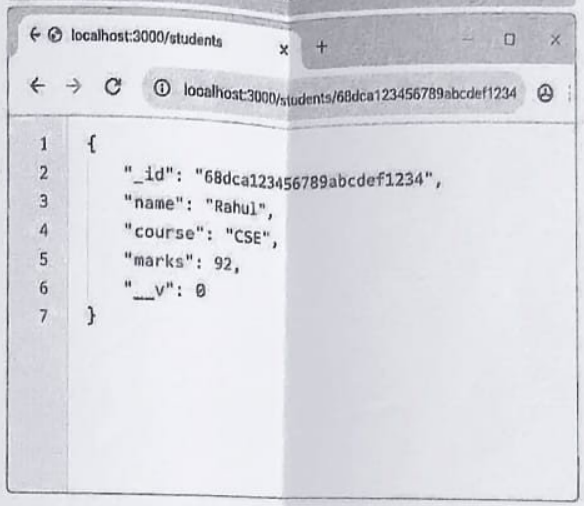
3)
• catch(error => {
  console.log(error);

output:

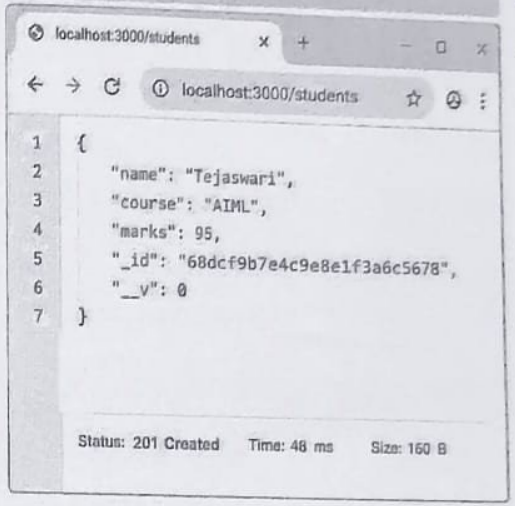
1. GET All Students (http://localhost:3000/students)



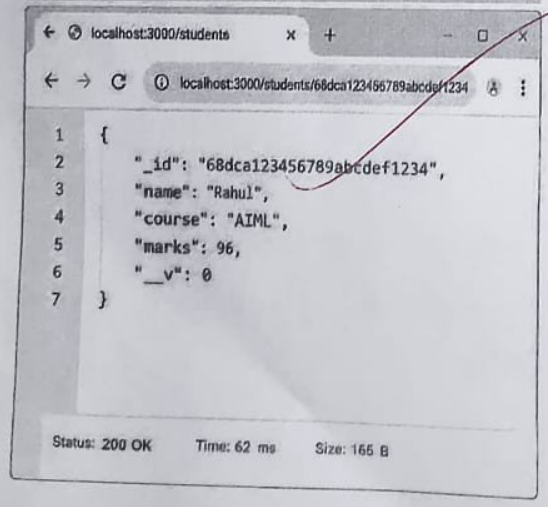
2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



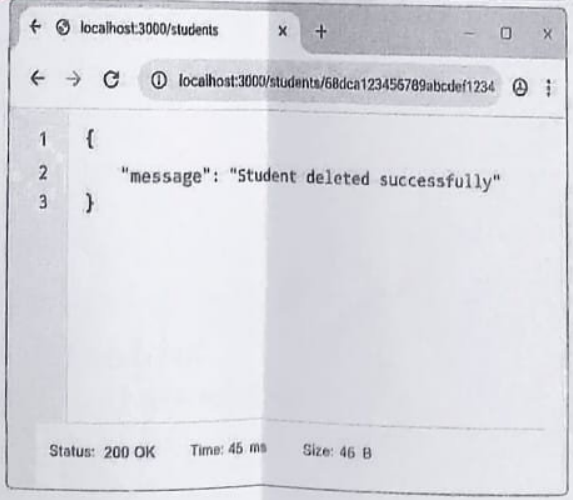
3. POST (Create Student) – Using Postman



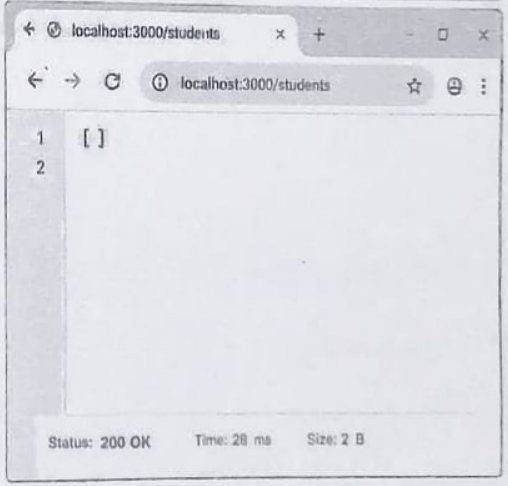
4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)



3)

requests.

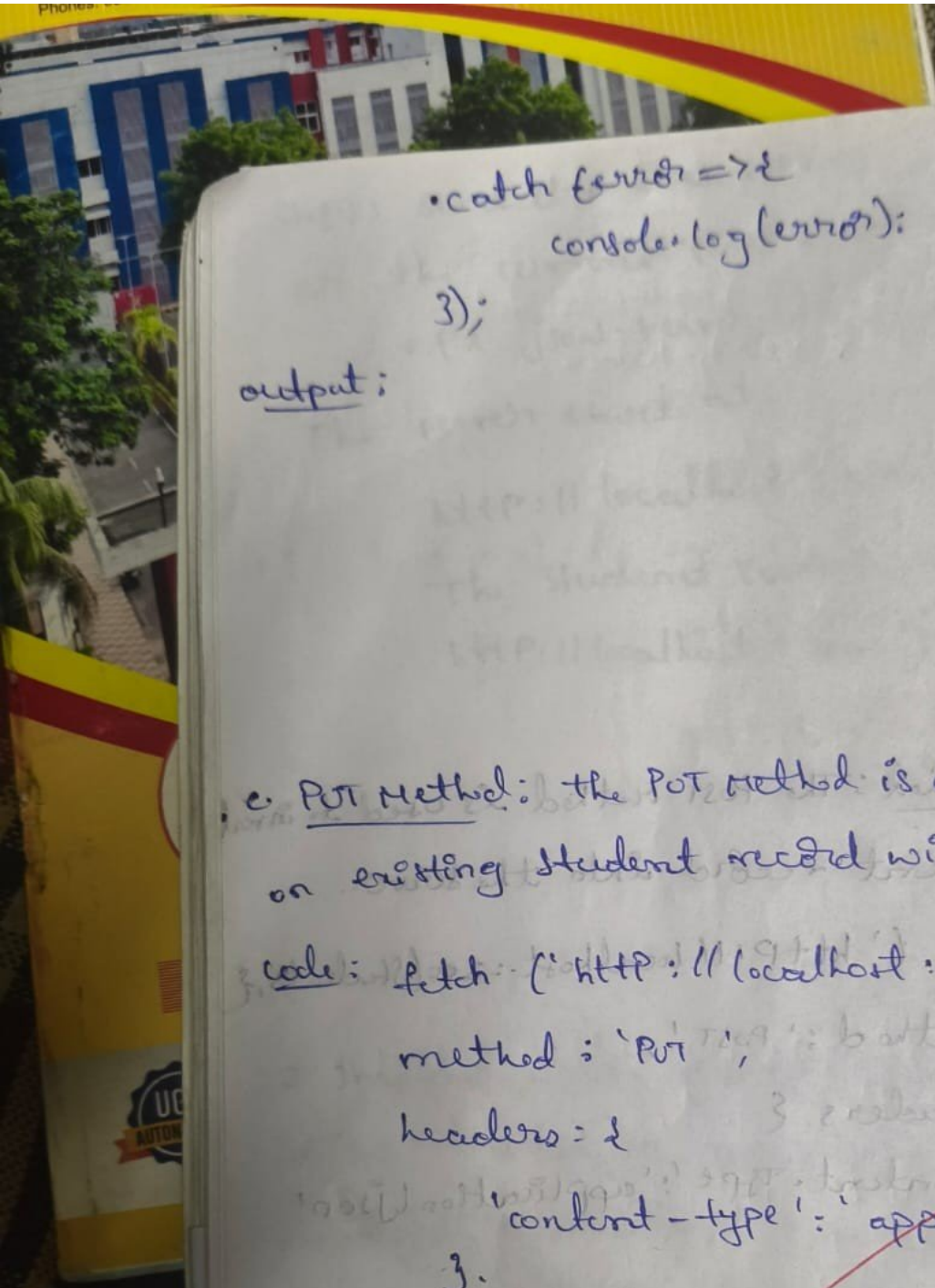
```
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:

D. PATCH Method: the PATCH method is used to update only selected fields of an existing student record.

code:

```
fetch('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'Content-Type': 'application/json',
    },
    body: JSON.stringify({
        marks: 96
    })
})
.then(response => response.json())
.then(data => {
```



```
.catch (error => {
    console.log(error);
});
```

output:

c. PUT Method: the PUT method is used to replace an existing student record with update data

code:

```
fetch ('http://localhost:3000/students/1',{
    method : 'PUT',
    headers : {
        'content-type' : 'application/json',
    },
    body : JSON.stringify({
        name : 'Ashok',
        course : 'AIML',
        marks : 95
    })
})
.then (response => response.json())
```

output:

D. PATC

update

record

code :

Step 3: Start the JSON server

for the current JSON server version,

```
npx json-server db.json
```

The server start at:

http://localhost:3000

The student records can be accessed

http://localhost:3000/students

2. Implementation of HTTP methods

a. GET method: The GET method is used to re

student records from JSON server

code:

```
fetch ('http://localhost:3000/students')
.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log(error);
});
```

```
mkdir JSON server
cd JSON server
npm init -y
npm install JSON server
```

Step2: create db.json

create a file named db.json and add the following student data:

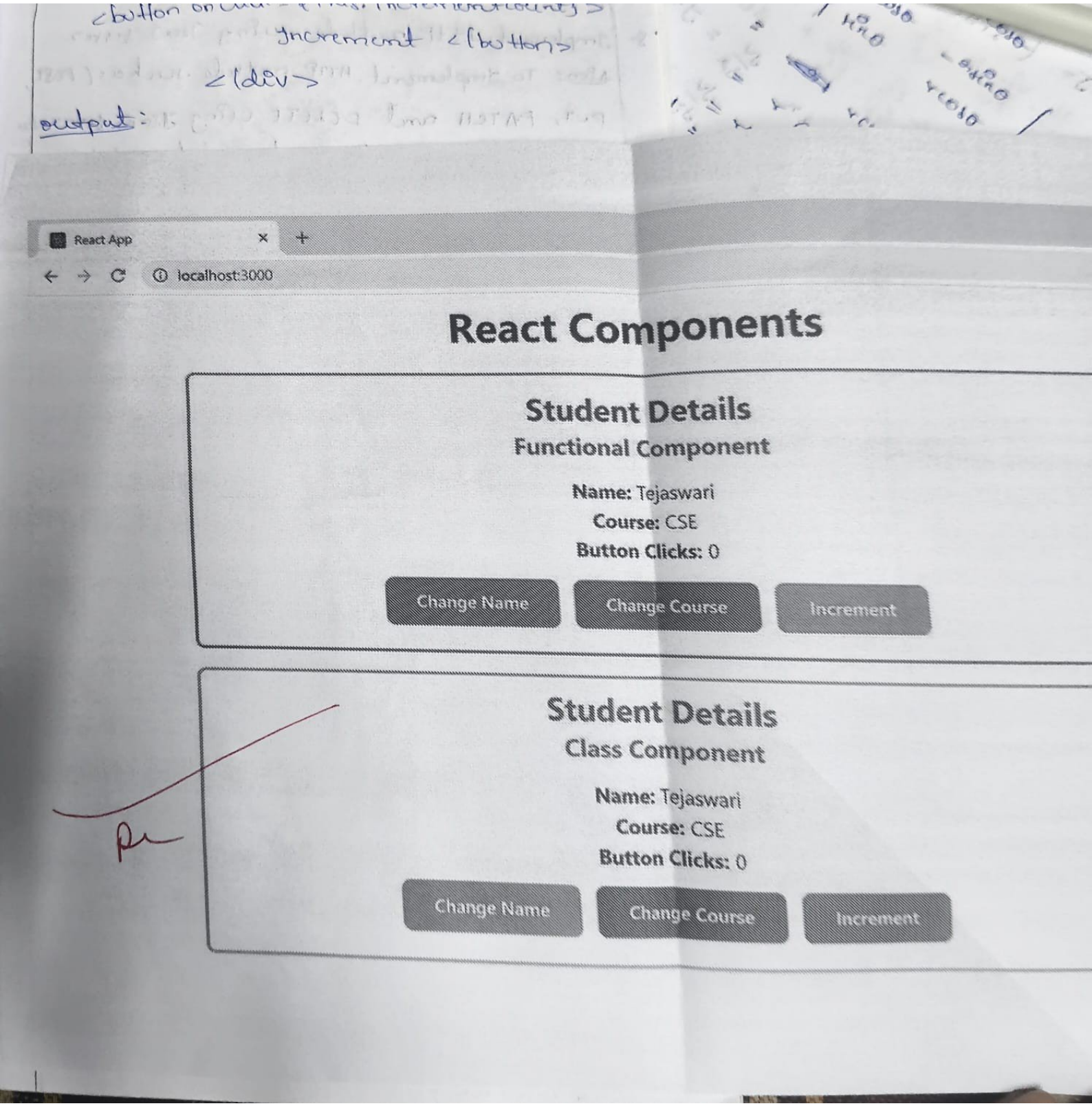
```
{
  "Students": [
    {
      "id": "1",
      "name: "Ashok",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2"
      "name": "AB",
      "course": "CSDA",
      "marks": 90
    },
    {
      "id": "3"
      "name": "BC"
      "course": "IT"
      "marks": 88
    }
```

mkdir JSON server
cd JSON server
npm init -y
npm install JSON server

Step2: create db.json

create a file named db.json and add the following student data:

```
{
  "Students": [
    {
      "id": "1",
      "name: "Ashok",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2"
      "name": "AB",
      "course": "CSE",
      "marks": 90
    },
    {
      "id": "3"
      "name": "BC",
      "course": "IT",
      "marks": 88
    }
```


Increment
output:
React App
localhost:3000
React Components
Student Details
Functional Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment
Student Details
Class Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

```
increment count
    this.setState((PrevState) => {
        count: PrevState.count + 1
    }));
};
render(){
    return(
        <div style={{
            border: "2Px solid green",
            Padding: "20Px",
            margin: "20Px",
            borderRadius: "10Px",
            textAlign: "center"
        }}>
        <h1> Student Details </h1>
        <h2> Class Component </h2>
<P><b>Name: </b>{this.state.name}</P>
<P><b>Course: {this.state.course}</P>
<P><b>Button clicks: </b>{this.state.count}</P>
<button onClick={this.changeName}>
    changeName </button>
<button onClick={this.changeCourse}>
    change course </button>
```

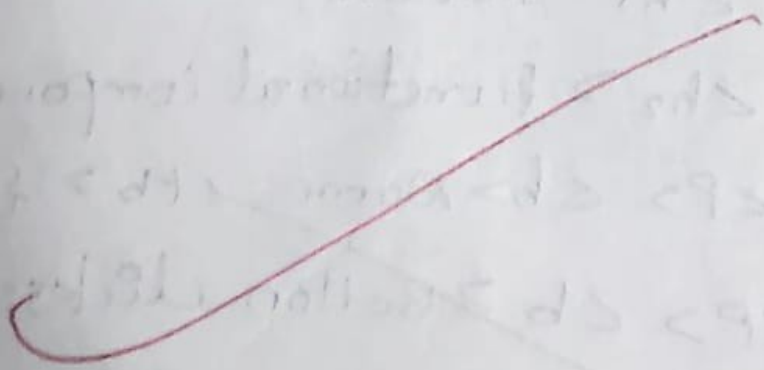
. class component:

A class component is an ES6 Javascript class that extends React.component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React {Component} from
'react':
class Studentclass extends Component {
    constructor (Props){
        super (Props);
        this.state = {
            name = "Ashok",
            course = "computer science"
            count = 0
        };
    }
    changeName = () => {
        this.setState ({
            name: "Ashokk"
        });
    };
    changeCourse = () => {
        this.setState ({
            course: "CSM"
        });
```

output: When the User the buttons, the name course and count change dynamically without reloading the webpage.



```
const changeCourse = () => {
    setCourse("CSD");
};
const incrementCount = () => {
    setCount(count++);
};
return (
  <div style={{
        border: "8px solid blue",
        padding: "20px";
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1>Student details</h1>
    <h2>Functional Component</h2>
    <p><b>Name:</b> {name}</p>
    <p><b>Button clicks:</b> {count}</p>
    <button onClick={changeName}>
        changeName</button>
    <button onClick={changeCourse}>
        change course</button>
    <button onClick={incrementCount}>
        Increment</button>
  </div>
);
```

EXPERIMENT - 7

* Create a Functional component and class component using react.

Aim: To develop a react application using Functional components and class components to display Student details, manage state, handle button clicks and dynamically update data on a web

· Functional component:

A Functional component is a javascript function that returns JSX to display the UI Interface. It uses react Hooks such as useState to manage state and update the webpage dynamically.

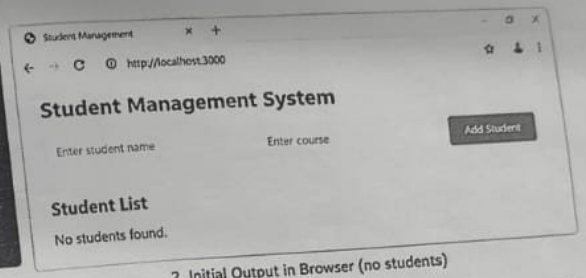
Program:

```
import react, {useState} from
'react':
function StudentFunctional(){
    const [name, setName] = useState("Ashok");
    const [course, setCourse] = useState("CSE");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Ashokk");
    };
```

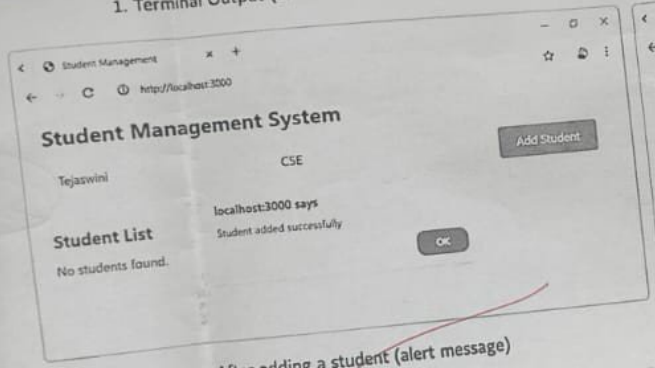
```
const changeCourse
    set course(
};
const increm
    set cou
};
return (
  <div sty
    bac
    Pad
    mar
    bor
    ten
  >
  <h1>
  <h2>
  <P> <
  <P> <
  <button
  < button
  <button
  </div
);
```



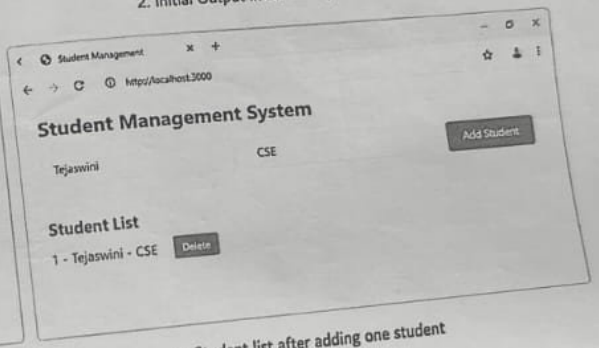
1. Terminal Output (after running `node server.js`)



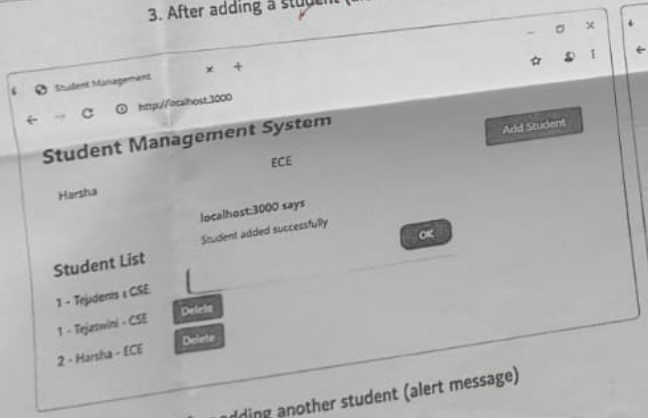
2. Initial Output in Browser (no students)



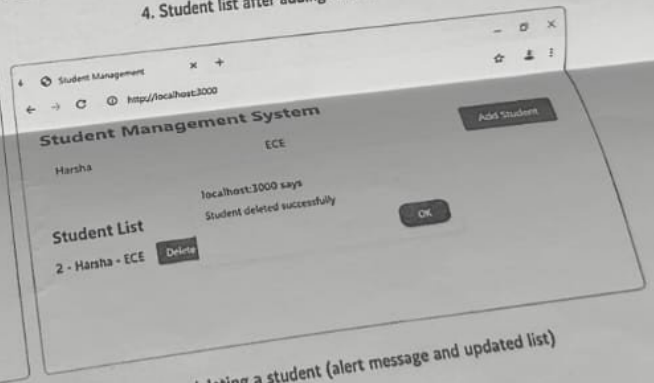
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

output :

Student management system

Enter student name

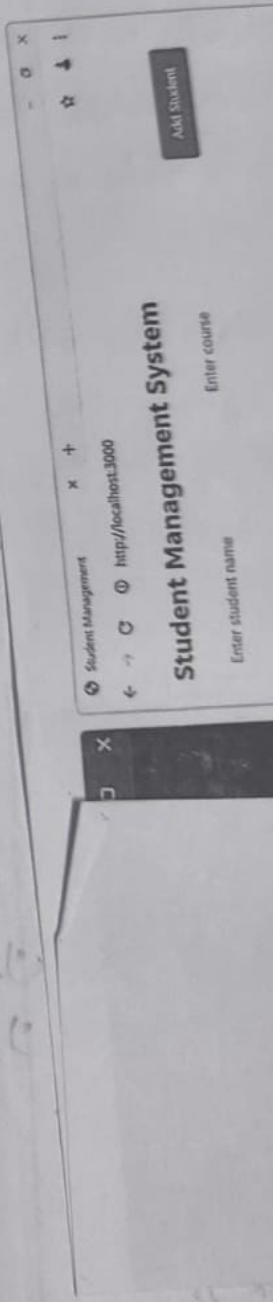
Enter course

Add Student

Student List :-

1. Manohar – CSM [delete]
2. Manoj – CSE [delete]

• the Express.js server is successfully start
(at http://localhost:3000
the browser displays the student
management system with fields.



```
students.forEach(student => {
    const li = document.createElement("li");
    li.innerHTML = student.id + " - " +
        student.name + " - " +
        student.course + " <button
onclick='deleteStudent(" + student.id + ")'>
        Delete </button>";
    list.appendChild(li);
  });
});
}

function deleteStudent(id) {
  fetch("/students/" + id,
    { method: "DELETE" })
  .then(response => response.json())
  .then(data => {
    alert(data.message);
    displayStudents();
  });
}
displayStudents();
</script>
</body>
</html>
```

```
fetch("/students", {
    method: "POST",
    headers: {
        "content-Type": "application/json"
    },
    body: JSON.stringify({
        name: name,
        course: course
    })
})
.then(response => response.json())
.then(data => {
    alert(data.message);
    document.getElementById("name").value = "";
    document.getElementById("course").value = "";
    displayStudents();
});
}
function displayStudents() {
    fetch("/students").then(response =>
        response.json()).then(students => {
    let list = document.getElementById(
```

Public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management
</head>
<body>
    <h1> Student management system
    <input type="text" id="nam
    Placeholder=" Enter student na
    <input type="text" id="cour
    Placeholder=" Enter course ">
    <button onclick="addStudent(
    Add student </button>
    <h2> Student list </h2>
    <ul id="studentList"></ul>
<script>
    function addStudent(){
    let name= document.getElement
        Id("name").val
    let course =document.getEleme
        ById("course").va
```

```
//Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push(student);
    res.json({ message: "Student added successfully",
        student: student });
});

// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student =>
        student.id !== id);
    res.json({
        message: "Student deleted successfully"
    });
});

// Start server
app.listen(PORT, () => {
    console.log("Server running at http://
        localhost:${PORT}");
});
```

3. Using Built-in Core modules:

Node.js also comes equipped with dozens of built-in modules (like fs for file systems & path for file routing) that don't need installation.

we will load them by module names:

```
// common js style
const fs = require('fs');
// Es modules style
import fs from 'fs';
```

Here are some of the most commonly used ones:

1. fs = file system operations
2. http = HTTP server and client
3. Path - file path utilities
4. os - operating system utilities
5. events - event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL query string handling

2. the ES modules method (modern standard),

ES modules use export to share code and import to use it to make node.js recognize ES modules, You must either use the .mjs file extension or add "type": "module" to your package.json file.

a. create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercale";
export default calculatorName;
```

b. use the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

output: supercale

5. Implement modules in Node.js

In Node.js you can implement modules using two distinct systems. common JS (js) which is traditional default, and ES modules (ESM) the modern javascript standard.

1. the common JS method (default):

common js uses module.exports to share code and require() to import it. This system loads modules synchronously.

create the module (math.js):

```
const add = (a, b) => a+b
const subtract = (a, b) => a-b
module.exports = {
    add,
    subtract
};
```

b use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

output: 15
5